

U-MV: Regional-scale *Acacia tortilis* crown mapping from UAV imagery

Technical hand-over and reproduction guide

Software, environment, data, training, evaluation and regional inference

Document version 1.2 · 09 September 2026

Repository: github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping

Reference: Gibril et al. (2026), Int. J. Appl. Earth Obs. Geoinf. 148, 105214, doi:10.1016/j.jag.2026.105214

Prepared by Mohamed Barakat A. Gibril

GIS and Remote Sensing Center, Research Institute of Sciences and Engineering, University of Sharjah

Contact: mbgibril@sharjah.ac.ae

The dataset described herein is not publicly shareable and is provided for project use only.

Contents

1 Read this first	5
1.1 What you receive	5
1.2 How to use this document	5
1.3 Conventions	5
2 Setup procedure (Windows, from scratch)	6
Step 1 — Prerequisites	6
Step 2 — Download the code	7
Step 3 — Copy the archive to a local disk	7
Step 4 — Give Docker enough memory	7
Step 5 — Confirm Docker can use the GPU	8
Step 6 — Tell Docker where the data is	8
Step 7 — Build the image	8
Step 8 — Start the container	9
Step 9 — Verify the software	9
Step 10 — Verify the dataset	9
Step 11 — Identify the checkpoint	9
Step 12 — Reproduce the published accuracy	9
Step 13 — Map an orthomosaic (optional)	10
Step 14 — Leave and come back	10
Alternative: the WSL2 Ubuntu terminal	10
If something goes wrong	10
3 Framework overview	11
3.1 Highlights	11
3.2 Repository layout	12
3.3 Published accuracy	12
4 Software environment and installation	13
4.1 Software stack and version rationale	13
4.2 Docker route (recommended)	13
4.3 Manual route (conda)	14
4.4 Verification	15
5 Data	16
5.1 Source data	16
5.2 Tiles and masks	16
5.3 Integrity check	16
5.4 Export from ArcGIS Pro (summary)	17
5.5 Location of the dataset	17

5.6 Orthomosaics for regional inference	17
6 Training	18
6.1 Model and optimisation settings	18
6.2 Commands	18
6.3 Expected resources (paper, TITAN RTX 24 GB, batch 2)	19
6.4 Practical notes	19
7 Evaluation	20
7.1 Metrics	20
7.2 Commands	20
7.3 Reference results (paper, Table 1)	20
8 Regional inference on orthomosaics	21
8.1 Processing steps	21
8.2 Commands	21
8.3 Parameters	21
8.4 Channel order	22
8.5 Outputs	22
8.6 Throughput	22
9 Reproducibility and paper-to-code mapping	23
9.1 Mapping of Section 3 of the paper to the configuration	23
9.2 Reconciliation with the original training logs	23
9.3 What is and is not fixed by the pinned environment	24
9.4 Archival	24
10 Troubleshooting	25
A Revision notes (September 2026)	26
A.1 Defects found in the previous revision	26
A.2 Structural changes	27
A.3 Compatibility with the archived work directories	27
A.4 Validation performed in this revision	27
A.5 Not validated here (requires a GPU host)	27
A.6 Resolved from the recovered work directories (September 2026)	27
A.7 Decisions left to the authors	28
B Pretrained weights	29
B.1 Provenance (verified September 2026)	29
B.2 Original work directories	29
C Command reference	30
C.1 Windows (PowerShell, in D:\U-MV)	30

C.2 Environment (Linux / WSL2) 30

C.3 Data and checkpoints 30

C.4 Training 30

C.5 Evaluation 30

C.6 Regional inference 30

C.7 Orthomosaic preparation 30

1 Read this first

This document accompanies the hand-over of the U-MV (U-shaped MambaVision) framework for *Acacia tortilis* crown mapping. It is written for a team member who has (i) the public code repository and (ii) read access to the project archive `A.tortilis_Data & Model` on the shared drive, which holds the dataset and the original MMSegmentation work directories with the trained checkpoints. Following it, that person installs the software, verifies it, reproduces the published accuracy figures and produces regional crown maps from new UAV orthomosaics without further assistance.

1.1 What you receive

Item	Where	What it is
Code	https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping	model, configs, Docker build, tools, tests, this guide
Dataset	<code>Z:\...\A.tortilis_Data & Model\Data</code> used to build the model	<code>img_dir/</code> and <code>ann_dir/</code> with <code>train</code> , <code>val</code> , <code>test2</code> , <code>Generalizability</code> (1024×1024 tiles, ~32 GB)
Trained models	<code>Z:\...\A.tortilis_Data & Model\A.tortilis_Models\Pretrained weights</code>	three MMSegmentation work directories with checkpoints, training configs and logs (~2.5 GB)

The archive also contains `A.tortilis_Models\MMSegmentation_Folder`; it is the superseded container workspace and is **not** needed.

1.2 How to use this document

- **Chapter 2 is the procedure.** Fourteen numbered steps take an empty Windows workstation to the reproduced test metrics and a mapped orthomosaic: each step names where its commands run, gives them, and states the expected result. Work through chapter 2 alone; consult the rest only when a step refers to it.
- Chapter 3 describes the framework. Chapters 4 to 8 are the reference behind the steps: installation, data, training, evaluation and regional inference, including the Linux/WSL2 forms of every command.
- Chapter 9 maps the paper to the configuration files and records the reconciliation with the original training logs. Chapter 10 lists failure modes with remedies.
- The appendices record what changed in the code revision of September 2026, describe the released weights, and give a one-page command reference.

1.3 Conventions

PowerShell commands run in a Windows PowerShell window in the repository folder (`D:\U-MV`). Commands marked "inside the container" run in the Linux shell started by step 8, where the dataset is mounted at `/data` and the checkpoint folder at `/weights`. Paths on the host are examples and are adapted where the procedure says so.

2 Setup procedure (Windows, from scratch)

This chapter is the complete procedure for the receiving team member. It starts from an empty Windows 11 workstation with an NVIDIA GPU, read access to the project share (Z:), and Windows PowerShell. Fourteen steps, in order. Each step says where the commands run (PowerShell or inside the container), gives the commands, and states what a successful result looks like. Commands are copied line by line; a line ending in a backslash continues on the next line and is pasted together with it. The other chapters are reference material and are not needed to complete this one.

Overview (steps 1 to 8 are done once; steps 9 to 12 verify the installation):

Step	What	Where	Time
1	Prerequisites: driver, Docker Desktop, Git	Windows	10 min
2	Download the code (<code>git clone</code>)	PowerShell	1 min
3	Copy the archive from Z: to a local disk	PowerShell	10–60 min
4	Give Docker enough memory	PowerShell	2 min
5	Confirm Docker can use the GPU	PowerShell	1 min
6	Tell Docker where the data is (<code>docker\.env</code>)	PowerShell	1 min
7	Build the image	PowerShell	30–60 min
8	Start the container	PowerShell	1 min
9	Verify the software	container	2 min
10	Verify the dataset	container	1 min
11	Identify the checkpoint	container	1 min
12	Reproduce the published accuracy	container	30 min
13	Map an orthomosaic (optional)	container	varies
14	Leave and come back	both	–

Facts to know before starting:

- **Which weights.** `best_mIoU_iter_*.pth` in each work directory (the checkpoint with the best validation mIoU, used for the paper's test results): tiny `best_mIoU_iter_100000.pth`, small `best_mIoU_iter_95000.pth`, base `best_mIoU_iter_60000.pth`. Every tool accepts the folder instead of the file and selects it automatically. The files on GitHub are byte-identical copies; Git LFS is not needed.
- **Which model.** Start with U-MV-small; it produced the regional maps and has the best generalisability figures.
- **Dataset.** The archive holds 4 893 (train), 2 407 (val), 3 123 (test2) and 2 162 (Generalizability) tiles of 1024×1024 pixels. The published models were trained on 26 615 tiles; the training folder was pruned afterwards. Evaluation and inference are exact; retraining reproduces the recipe, not the model.
- **Confidentiality.** The dataset is not publicly shareable.

Step 1 — Prerequisites

Windows · once per machine

Need	How to get or check it
NVIDIA driver ≥ 520	open PowerShell (Start → type PowerShell → Enter) and run <code>nvidia-smi</code> ; a table with the GPU name and driver version must appear. Otherwise install the driver from nvidia.com
Docker Desktop	install from docker.com and start it; in Settings → General keep "Use the WSL 2 based engine". A steady whale icon in the tray means the engine runs

Need	How to get or check it
Git	<code>git --version</code> in PowerShell; otherwise install Git for Windows (git-scm.com) with the default options
Disk	about 40 GB free on the local data disk (D: below) and 40 GB on C: for the Docker image
RAM	Task Manager → Performance → Memory: note the total (needed in step 4)
Internet	needed for steps 2, 5, 7 and 9

Use one PowerShell window for the whole procedure. Commands marked "inside the container" (steps 9 to 13) are typed in the same window after step 8.

Step 2 — Download the code

PowerShell · once · the repository is public

```
cd D:\
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git U-MV
cd D:\U-MV
git log -1 --format=%cd
```

You should see: a Cloning into 'U-MV'... message and then the date of the latest commit. The code is now in D:\U-MV. *If a copy from an earlier attempt exists* (any folder that already contains docker\Dockerfile), rename or delete it first, or open it and run `git pull` instead of cloning. Any other location works; use it wherever D:\U-MV appears below.

Step 3 — Copy the archive to a local disk

PowerShell, in D:\U-MV · once · 10 to 60 minutes

Tiles must not be read from the share (too slow). The copy goes to a short path **without** the & character; D:\A.tortilis_Data_Model is used everywhere below. One command does the copy, checks the tile counts and prints the two lines needed in step 6:

```
tools\windows\copy_archive.cmd D:\A.tortilis_Data_Model
```

You should see: two robocopy summaries with Failed 0, then the tile counts marked ok (train 4893, val 2407, test2 3123, Generalizability 2162), three best_mIoU_iter_*.pth paths, and RESULT: OK. *If the archive was copied earlier* to another folder whose name contains no &, keep it and skip this step; note its path for step 6. *If the source is not found*, connect the Z: drive (File Explorer → This PC) and run the command again. The script can be run again at any time; it only copies what is missing. (It runs tools\windows\copy_archive.ps1, which can also be given -Source if the share is mounted under another letter.)

Step 4 — Give Docker enough memory

PowerShell · once

Docker Desktop runs inside WSL2, and the image build compiles software that needs several GB. Set the WSL2 limit below the machine's total RAM: 24GB on a 32 GB machine, 40GB on 64 GB.

```
notepad $env:USERPROFILE\.wslconfig
```

Notepad asks whether to create the file: answer Yes. Make its content exactly the following, save with Ctrl+S, close Notepad:

```
[wsl2]
memory=24GB
swap=8GB
```

Apply the setting (Docker Desktop restarts its engine; wait until the tray icon is steady again, about one minute):

```
wsl --shutdown
```

Step 5 — Confirm Docker can use the GPU

PowerShell, in D:\U-MV · once

```
docker\umv.cmd gpu
```

You should see: the same GPU table as `nvidia-smi`, printed from inside a container (the first run downloads a small test image). Note the GPU name for step 7. *If it fails:* Docker Desktop → Settings → Resources → WSL integration → enable the default distribution → Apply & restart, then repeat.

Step 6 — Tell Docker where the data is

PowerShell, in D:\U-MV · once

```
copy docker\.env.windows.example docker\.env
type docker\.env
```

You should see: the file content, whose two path lines are

```
DATA_DIR="D:/A.tortilis_Data_Model/Data used to build the model"
WEIGHTS_DIR="D:/A.tortilis_Data_Model/A.tortilis Models/Pretrained weights"
```

These are correct if step 3 used D:\A.tortilis_Data_Model. Otherwise run `notepad docker\.env` and replace the two paths with the ones printed at the end of step 3 (forward slashes, quotes kept), save, close.

Step 7 — Build the image

PowerShell, in D:\U-MV · once · 30 to 60 minutes

Find the architecture number of the GPU named in step 5:

GPU	Number
TITAN RTX, RTX 2060 / 2070 / 2080, Quadro RTX 4000–8000	7.5
RTX A4000 / A5000 / A6000, RTX 3060 / 3070 / 3080 / 3090	8.6
A100	8.0
RTX 4070 / 4080 / 4090, RTX 6000 Ada	8.9

Then build with that number (7.5 is shown; replace it if different):

```
docker\umv.cmd build 7.5
```

Leave the window open and the computer awake. The stages appear in this order: base image download (sha256: lines), apt packages, conda packages, the MMCV compilation (long, few visible lines), mamba-ssm, the project install.

You should see: Build finished. Next: `docker\umv.cmd shell`. To double-check, `docker images umv` prints one line with `umv` and `latest`.

cannot allocate memory: raise memory= in step 4, run `wsl --shutdown`, run the build command again (finished stages are reused). *Any other ERROR:* copy the last 60 lines of the window and send them to the project lead. On a machine with 64 GB of RAM the build can use four compiler jobs: `docker\umv.cmd build 7.5 4`.

Step 8 — Start the container

PowerShell, in D:\U-MV · every session

```
docker\umv.cmd shell
```

You should see: the prompt changes to `root@...:/workspace/U-MV#`. You are now inside Linux; steps 9 to 13 are typed here. Check the mounts:

```
ls /data
ls /weights
nvidia-smi
```

You should see: `ann_dir img_dir`; three `mambavision-*` folders; the GPU table. If `/data` or `/weights` is empty: type `exit`, correct `docker\.env` (step 6), run step 8 again.

Step 9 — Verify the software

inside the container · once

```
python tools/verify_install.py --variant small
```

The first run downloads the MambaVision-S encoder (about 200 MB) from the Hugging Face Hub.

You should see: every library with a version, CUDA available: True with the GPU name, `mamba_ssm selective_scan_fn: ok, mmcv.ops ... ok, forward (1, 3, 512, 512) -> (1, 2, 512, 512)`, and `RESULT: OK`.

Step 10 — Verify the dataset

inside the container · once

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
```

You should see: the four tile counts of step 3, mask values `[0, 1]`, and `RESULT: OK`.

Step 11 — Identify the checkpoint

inside the container · once

```
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
```

You should see: `"path": ".../best_mIoU_iter_95000.pth", "variant": "small", "iteration": 95000`.

Step 12 — Reproduce the published accuracy

inside the container · once · 10 to 20 minutes per run

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability
```

Each run ends with a metric table.

You should see: on `test2`, `mIoU` ≈ 85.4 and `mFscore` ≈ 91.6 (paper: 85.38 / 91.58); on `Generalizability`, ≈ 89.5 and ≈ 94.2 (paper: 89.48 / 94.17). Send both tables to the project lead. The installation is now verified.

Optional, for the other two models:

```
CKPT_T="/weights/mambavision-t_generic-unet_acacia"
python tools/test.py configs/mambavision/U-MV-tiny.py "$CKPT_T" --test-split test2
CKPT_B="/weights/mambavision-b_generic-unet_acacia"
python tools/test.py configs/mambavision/U-MV-base.py "$CKPT_B" --test-split test2
```

(paper: tiny 85.44 / 91.61, base 85.30 / 91.52).

Step 13 — Map an orthomosaic (optional)

inside the container · per orthomosaic

In Windows, create the folder `orthos` inside `D:\A.tortilis_Data_Model\Data` used to build the model and put a GeoTIFF orthomosaic in it (any name; `site.tif` is used below). Then:

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/geospatial_inference.py \
    --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
    --input /data/orthos/site.tif --output /data/predictions/site_crowns.gpkg \
    --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob
```

(The command is one bash command written over four lines; the trailing backslashes join them. Paste all four lines together.)

You should see: progress bars and a JSON summary with the number of polygons. In Windows the result is `...\Data used to build the model\predictions\site_crowns.gpkg`; it opens in ArcGIS Pro or QGIS on top of the orthomosaic, with `area` and `mean_prob` attributes. For a folder of orthomosaics use `tools/batch_geospatial_inference.py` with `--input-dir` and `--output-dir` (see the inference chapter).

Step 14 — Leave and come back

both

`exit` leaves the container. The image stays built; next time only step 8 is needed (`cd D:\U-MV` then `docker\umv.cmd shell`). To receive code updates, run `git pull` in `D:\U-MV` before step 8; the container sees the new files immediately. Training outputs go to `D:\U-MV\work_dirs`, predictions to wherever `--output` points.

Alternative: the WSL2 Ubuntu terminal

The same steps work from an Ubuntu (WSL2) terminal with Linux commands: the archive is visible as `/mnt/d/A.tortilis_Data_Model/...`, `nano` replaces `notepad`, `docker/.env.example` replaces `docker/.env.windows.example`, and the `docker compose ... build` and `run --rm umv` commands of the installation chapter replace `docker\umv.cmd`. Nothing inside the container changes.

If something goes wrong

Consult the troubleshooting chapter first. When reporting a problem, give the step number, the exact command, the last 60 lines of output and, for steps 9 onward, the output of `python tools/verify_install.py`.

Contact: Mohamed Barakat A. Gibril (mbgibril@sharjah.ac.ae).

3 Framework overview

U-MV couples the hierarchical MambaVision encoder of Hatamizadeh and Kautz (2025), a hybrid of Mamba state-space mixers, self-attention and convolution, with a lightweight U-Net-style decoder. The encoder produces feature maps at strides 4, 8, 16 and 32; the decoder upsamples the deepest map stage by stage, concatenating skip features and applying two 3×3 convolution–BatchNorm–ReLU blocks per stage, and a 1×1 convolution yields two-class logits. Three encoder sizes are used: tiny (80/160/320/640 channels), small (96/192/384/768) and base (128/256/512/1024). Training minimises cross-entropy plus three times the Dice loss on 1024×1024 UAV tiles at 2.5 cm ground sampling distance.

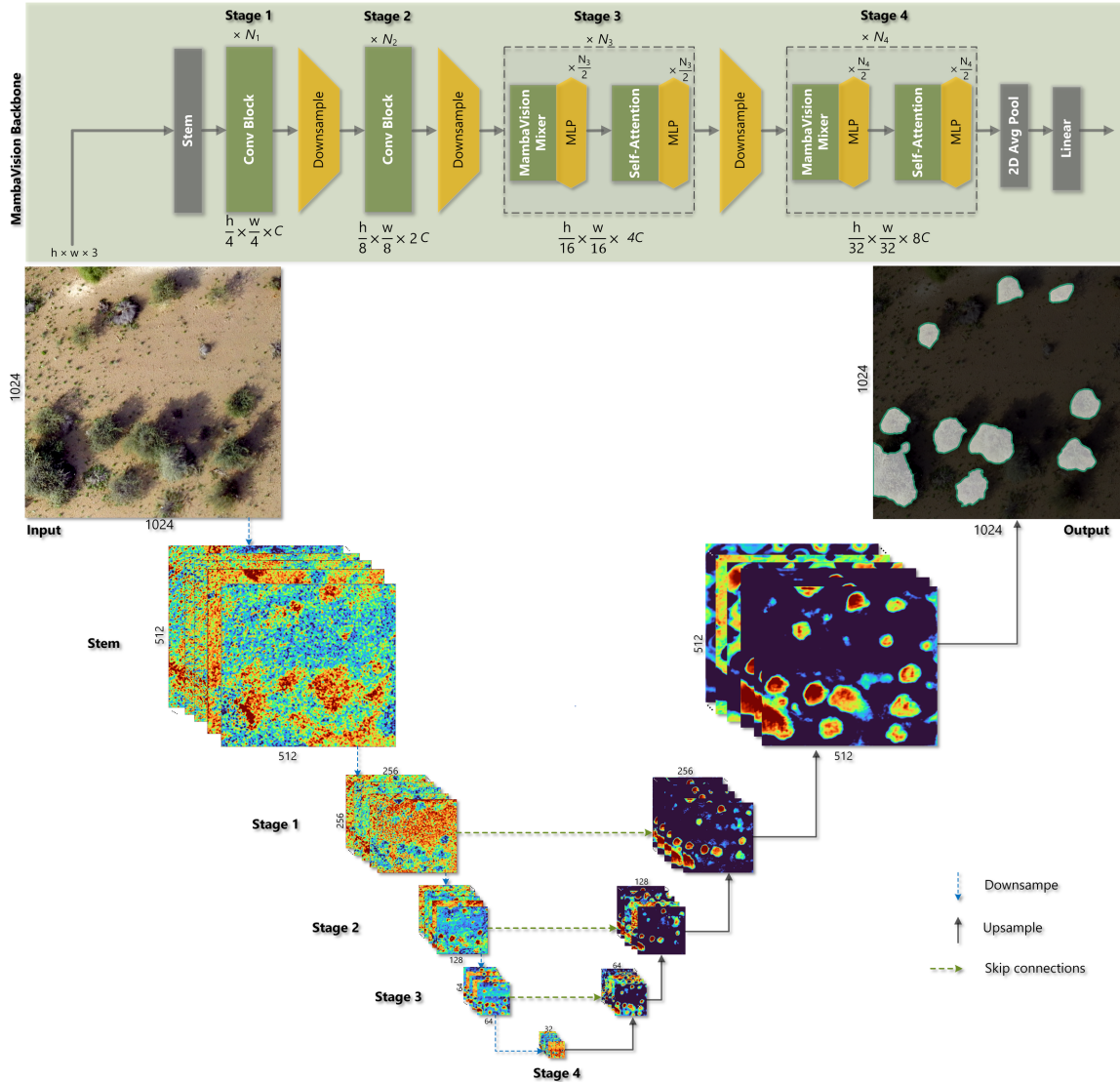


Figure 1. U-MV architecture: four-stage MambaVision encoder (top) and U-Net decoder with skip connections applied to a 1024×1024 UAV tile.

3.1 Highlights

- **Three variants** (tiny / small / base; 35.4 M and 54.2 M parameters for tiny and small) with released checkpoints; test-set mIoU 85.30–85.44 % and mF-score 91.52–91.61 %.
- **Self-contained MMSegmentation extension:** `pip install -e .` registers the backbone, decoder and dataset; no files are copied into MMSegmentation.

- **Reproducible environment:** Dockerfile pinned to the original stack (PyTorch 2.6.0 + CUDA 11.8, MMCV 2.1.0, MMSegmentation 1.2.2, mamba-ssm 2.2.4).
- **Regional inference:** seam-free sliding-window prediction over gigapixel orthomosaics, streaming vectorisation to GeoPackage/Shapefile with per-crown area and mean_prob attributes.

3.2 Repository layout

```

umv/
├── models/mamba_vision.py      installable package (registered with MMSegmentation)
├── models/unet_head.py        MambaVisionBackbone (Hugging Face nvidia/MambaVision-*-1K)
├── datasets/uav_acacia.py      GenericUNetHead (lightweight U-Net decoder)
├── inference/                  UAVAcaciaDataset (background / acacia)
├── checkpoint.py              tiling, blending, vectorisation, pipeline, CLI options
├──                             checkpoint inspection and variant detection
configs/
├── _base_/                    models/umv_unet.py · datasets/uav_acacia_dataset.py ·
├──                             schedules/schedule_100k_adamw.py · default_runtime.py
├── mambavision/               U-MV-tiny.py · U-MV-small.py · U-MV-base.py
tools/
├── train.py, test.py          MMSegmentation v1.2.2 entry points (+ --test-split)
├── geospatial_inference.py    one orthomosaic -> crown polygons
├── batch_geospatial_inference.py folder tree -> crown polygons
├── verify_install.py, inspect_checkpoint.py, download_backbones.py, check_dataset.py
docker/                         Dockerfile · docker-compose.yml · run.sh
requirements/                   core.txt · geo.txt · mamba.txt · dev.txt
Pretrained_Weights/             released checkpoints (Git LFS) + README
tests/                          CPU unit tests (tiling, vectorisation, pipeline)
docs/                           01 installation ... 08 hand-over checklist, REVISION_NOTES

```

3.3 Published accuracy

Model	Validation mIoU	Validation mF-score	Test mIoU	Test mF-score
U-MV-t	87.91	93.20	85.44	91.61
U-MV-s	88.02	93.27	85.38	91.58
U-MV-b	88.11	93.32	85.30	91.52

Values in percent (paper, Table 1). On the generalisability set (~11 km², 2 165 tiles) U-MV-s reached 89.48 % mIoU and 94.17 % mF-score. Training on a TITAN RTX (24 GB) with batch size 2 took 9.33 h (tiny), 11.8 h (small) and 18.12 h (base) for 100 000 iterations.

4 Software environment and installation

Two routes are supported. The Docker route reproduces the environment in which the published experiments were executed and is the recommended option for hand-over to new team members. The manual route documents every dependency for users who cannot run containers.

4.1 Software stack and version rationale

Component	Version	Reason
Python	3.11	Version of the original container (docs/reference/environment.frozen.yml).
PyTorch / torchvision	2.6.0 + cu118 / 0.21.0	Original experiments; CUDA 11.8 wheels run on driver ≥ 520 (TITAN RTX, RTX A5000).
MMEEngine	0.10.7	Handles <code>torch.load(weights_only=True)</code> introduced in PyTorch 2.6.
MMCV	2.1.0 (full build)	Latest release accepted by MMSegmentation 1.2.2 (<code>mmcv</code> $\geq 2.0.0rc4$, $< 2.2.0$). No prebuilt wheel exists for PyTorch 2.6, so it is compiled from source.
MMSegmentation	1.2.2	Last stable 1.x release; equivalent to the <code>main</code> branch used originally.
transformers / timm	4.50.0 / 1.0.15	Versions with which the MambaVision remote code (<code>trust_remote_code=True</code>) is known to import (<code>timm.models.registry</code> , <code>timm.models.layers.shims</code>).
mamba-ssm	2.2.4	Provides <code>selective_scan_fn</code> , the CUDA kernel used by the MambaVision mixer. Requires <code>nvcc</code> .
GDAL / rasterio / geopandas	3.10 / 1.4 / 1.0 (conda-forge)	GDAL <code>Polygonize</code> is used for streaming vectorisation of gigapixel probability rasters.

MambaVision does **not** call `causal_conv1d`; that package is optional.

4.2 Docker route (recommended)

4.2.1 Prerequisites on Windows 11 / WSL2

- 1 Install an NVIDIA driver ≥ 520 on Windows (the driver is shared with WSL2; do not install a driver inside WSL2).
- 2 Install Docker Desktop with the WSL2 backend and enable the Ubuntu distribution under *Settings* → *Resources* → *WSL integration*.
- 3 Verify GPU visibility from WSL2:

```
docker run --rm --gpus all nvidia/cuda:11.8.0-base-ubuntu22.04 nvidia-smi
```

- 4 Grant WSL2 enough memory for the 1024×1024 training tiles (in `%USERPROFILE%\wslconfig`):

```
[wsl2]
memory=48GB
swap=16GB
```

then `wsl --shutdown` from PowerShell.

4.2.2 Build

```
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git
cd U-MV-Acacia-tortilis-Crown-Mapping
git lfs install && git lfs pull      # released checkpoints; not needed with the project archive
cp docker/.env.example docker/.env && nano docker/.env    # DATA_DIR, WEIGHTS_DIR
docker compose --env-file docker/.env -f docker/docker-compose.yml build
```

The build compiles MMCV and mamba-ssm and typically takes 30–60 min. Two build arguments control the compilation: `CUDA_ARCH` (default "7.5;8.6", i.e. TITAN RTX and RTX A5000; pass only your GPU's value) and `MAX_JOBS` (parallel compiler jobs, default 4; each needs about 2 GB of RAM). On a machine where Docker/WSL2 has 16 GB or less:

```
docker compose --env-file docker/.env -f docker/docker-compose.yml build \
  --build-arg MAX_JOBS=2 --build-arg CUDA_ARCH="7.5"
```

`nvidia-smi` prints the GPU name; 7.5 = TITAN RTX / RTX 20xx, 8.6 = RTX A5000 / RTX 30xx, 8.0 = A100, 8.9 = RTX 40xx.

4.2.3 Windows (PowerShell + Docker Desktop)

`docker\umv.cmd` wraps the same compose commands for a PowerShell or cmd window, so that no long command has to be typed; `docker\.env.windows.example` is a ready-made `.env` with Windows paths (forward slashes, quoted):

```
copy docker\.env.windows.example docker\.env    # then edit the two paths if needed
docker\umv.cmd gpu                               # Docker can see the GPU?
docker\umv.cmd build 7.5                         # = compose build with MAX_JOBS=2, CUDA_ARCH=7.5
docker\umv.cmd shell                             # = compose run --rm umv
```

The complete Windows procedure, including copying the archive from the share with `tools\windows\copy_archive.cmd`, is `docs\08_handover_checklist.md`.

4.2.4 Run

```
cp docker/.env.example docker/.env              # set DATA_DIR and WEIGHTS_DIR (quoted if they contain spaces)
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
# inside the container
python tools/verify_install.py --variant small
```

The compose file bind-mounts the repository at `/workspace/U-MV`, the dataset at `/data`, the checkpoint folder read-only at `/weights`, a named volume for the Hugging Face cache and `work_dirs/` for training outputs. `docker/run.sh` offers the same without compose and reads the same `docker/.env`.

4.2.5 Offline use

The MambaVision code and ImageNet weights are downloaded once from the Hugging Face Hub into the `hf_cache` volume:

```
python tools/download_backbones.py --variants tiny small base
export HF_HUB_OFFLINE=1          # subsequent runs need no network
```

4.3 Manual route (conda)

Install in this order; later steps compile against earlier ones.

```
conda create -n umv python=3.11 -y && conda activate umv
# 1. PyTorch (CUDA 11.8 wheels)
pip install torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu118
# 2. MMEEngine and MMCV (compiled; requires nvcc from CUDA 11.8 toolkit on PATH)
pip install mmeengine==0.10.7
MMCV_WITH_OPS=1 FORCE_CUDA=1 TORCH_CUDA_ARCH_LIST="7.5;8.6" pip install --no-build-isolation mmcv==2.1.0
# 3. Mamba kernels
pip install packaging ninja && pip install --no-build-isolation -r requirements/mamba.txt
# 4. Geospatial stack (GDAL bindings from conda-forge)
conda install -c conda-forge gdal=3.10 rasterio=1.4 geopandas=1.0 shapely=2.1 pyproj=3.7 pyogrio -y
# 5. Project
pip install -r requirements/core.txt
pip install --no-deps -e .
python tools/verify_install.py
```

`pip install -e .` makes `umv` importable from any working directory; the scripts in `tools/` also work without installation because they add the repository root to `sys.path`.

4.4 Verification

```
python tools/verify_install.py           # imports, CUDA, mamba_ssm, mmcv.ops
python tools/verify_install.py --variant small # builds U-MV-small, forward pass, peak VRAM
python -m pytest tests -q                # tiling / vectorisation unit tests (CPU)
```

Expected: RESULT: OK, and for the second command a forward pass of a 512×512 tensor producing logits of shape (1, 2, 512, 512).

5 Data

5.1 Source data

The framework was developed on RGB orthomosaics acquired with a senseFly eBee X (S.O.D.A. camera) at ~122 m AGL, processed in Pix4Dmapper to a ground sampling distance (GSD) of 2.5–3 cm. Annotations were produced with the semi-automated workflow described in Section 3.1 of the paper (SAM-LoRA crown proposals, Mask2Former–Swin candidate detection, visual verification).

5.2 Tiles and masks

Orthomosaics and rasterised crown polygons were partitioned into 1024×1024 pixel image–mask pairs. This tile size was chosen so that a mean crown (~17.5 m², ~5 m diameter, ~28 000 pixels at 2.5 cm) is embedded in sufficient context (shadows, terrain transitions, neighbouring land cover).

Requirements for new data:

Item	Specification
Images	3-band RGB, 8-bit, .tif (GeoTIFF or plain TIFF). Other suffixes: set <code>img_suffix</code> in the dataset config.
Masks	Single-band 8-bit index raster, 0 = background, 1 = acacia, .tif (or .png with <code>seg_map_suffix= '.png'</code>). Not RGB, not 0/255.
Names	Identical basenames for image and mask within a split.
Size	1024 × 1024 recommended; other sizes work (the pipeline resizes to 1024 on the long side and random-crops 1024 × 1024 during training).
Ignore value	255 in a mask is ignored by the loss (<code>seg_pad_val=255</code>).

Directory layout (mounted at /data in the container):

```
/data
├── img_dir/
│   ├── train/          4 893 pairs in the archive (26 615 used for the published models)
│   ├── val/            2 407
│   ├── test2/          3 123 (in-distribution test)
│   └── Generalizability/ 2 162 (out-of-distribution test, separate region)
└── ann_dir/
    ├── train/ val/ test2/ Generalizability/
```

The archived training folder was pruned after the published runs (see the reproducibility chapter); the other three splits are complete.

A different root is selected without editing configs:

```
python tools/train.py configs/mambavision/U-MV-small.py \
  --cfg-options train_dataloader.dataset.data_root=/data/UAV_Acacia \
                 val_dataloader.dataset.data_root=/data/UAV_Acacia \
                 test_dataloader.dataset.data_root=/data/UAV_Acacia
```

5.3 Integrity check

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
```

reports pair counts, tile sizes, dtypes, mask values and ArcGIS side-car files (*.tif.aux.xml, *.tif.ovr) per split. Side-cars are created when tiles are opened in ArcGIS Pro; the loader ignores them (only files ending in .tif are listed), but they can be deleted from working copies to save space.

A split whose folder is not named test (for example test2) is evaluated without renaming: `python tools/test.py <config> <ckpt> --test-split test2`.

5.3.1 Manual check

```
import numpy as np, rasterio, glob, os
root = '/data'
for split in ['train', 'val', 'test2', 'Generalizability']:
    imgs = sorted(glob.glob(f'{root}/img_dir/{split}/*.tif'))
    anns = sorted(glob.glob(f'{root}/ann_dir/{split}/*.tif'))
    assert [os.path.basename(p) for p in imgs] == [os.path.basename(p) for p in anns], split
    with rasterio.open(imgs[0]) as im, rasterio.open(anns[0]) as an:
        assert im.count == 3 and im.dtypes[0] == 'uint8', im.profile
        assert an.count == 1 and set(np.unique(an.read(1))) <= {0, 1, 255}, an.profile
        assert (im.width, im.height) == (an.width, an.height)
    print(split, len(imgs), 'pairs OK')
```

5.4 Export from ArcGIS Pro (summary)

- 1 Rasterise the verified crown polygons to the orthomosaic grid (Polygon to Raster, cell size = orthomosaic cell size, value field = 1, NoData → 0).
- 2 Export image and mask tiles with the same fishnet (Split Raster, tile size 1024×1024 , format TIFF, no overlap for training tiles).
- 3 Convert masks to 8-bit unsigned (Copy Raster, pixel type 8_BIT_UNSIGNED) and verify with the script above.

5.5 Location of the dataset

The dataset folder can reside anywhere; its path is supplied once in `docker/.env` (`DATA_DIR`) or on the command line (`--cfg-options train_data_loader.dataset.data_root=...`). Prefer a local SSD over a network share, because training performs ~30 000 random tile reads per epoch. When copying, ArcGIS side-cars can be excluded:

```
robocopy "<source>\Data used to build the model" "D:\UAV_Acacia_Data" /E /XF *.aux.xml *.ovr *.xml /MT:16
```

If a network drive must be read directly from WSL2, mount it first (`sudo mkdir -p /mnt/z && sudo mount -t drvfs Z: /mnt/z`) and quote the path.

5.6 Orthomosaics for regional inference

Inference operates on whole orthomosaics. Convert them to Cloud-Optimised GeoTIFF (COG) once; windowed reads are then efficient from any storage:

```
gdal_translate -of COG -co COMPRESS=DEFLATE -co BLOCKSIZE=512 -co NUM_THREADS=ALL_CPUS in.tif out_cog.tif
```

6 Training

6.1 Model and optimisation settings

All settings are declared in `configs/` and inherited by the three variant files; nothing is hard-coded in Python.

Setting	Value	Config location
Encoder	MambaVision-T/S/B, ImageNet-1K initialisation (Hugging Face Hub)	<code>_base_/models/umv_unet.py</code> → <code>model.backbone</code>
Encoder widths	T (80, 160, 320, 640) · S (96, 192, 384, 768) · B (128, 256, 512, 1024)	<code>mambavision/U-MV-*.py</code>
Decoder	U-Net style, widths (256, 128, 64, 32), BN + ReLU, bilinear ×2 upsampling	<code>model.decode_head</code>
Loss	CE + 3 × Dice (Eq. 1 of the paper)	<code>decode_head.loss_decode</code>
Optimiser	AdamW, lr 1e-4, $\beta = (0.9, 0.999)$, weight decay 0.05; backbone lr multiplier 0.1 (effective 1e-5); no decay on normalisation layers	<code>_base_/schedules/schedule_100k_adamw.py</code>
Schedule	100 000 iterations, polynomial decay (power 0.9), validation every 5 000 iterations	same
Gradient clipping	L2 norm 0.01	<code>optim_wrapper.clip_grad</code>
Batch	2 × 1024 × 1024 tiles	<code>_base_/datasets/uav_acacia_dataset.py</code>
Augmentation	RandomResize (0.5–2.0), RandomCrop 1024 (<code>cat_max_ratio=0.75</code>), horizontal flip, photometric distortion	same
Checkpointing	every 5 000 iterations, plus <code>best_mIoU_iter_*.pth</code> on validation mIoU	<code>default_hooks.checkpoint</code>
Test-time inference	sliding window 1024 × 1024, stride 512	<code>model.test_cfg</code>

All three variants share this schedule; the training logs of the released checkpoints confirm it (`docs/07_reproducibility.md`, §7.2). Note that the archived training split is a pruned subset (4 893 of the 26 615 tiles used for the published models), so retraining from the hand-over folder will not reach the published accuracy exactly.

6.2 Commands

```
# inside the container, dataset mounted at /data
python tools/train.py configs/mambavision/U-MV-tiny.py
python tools/train.py configs/mambavision/U-MV-small.py
python tools/train.py configs/mambavision/U-MV-base.py
```

Useful options (inherited from MMSegmentation):

Option	Effect
<code>--work-dir work_dirs/umv_small_run2</code>	Output directory (default <code>work_dirs/<config-name></code>).
<code>--resume</code>	Continue from the latest checkpoint in the work directory.
<code>--amp</code>	Automatic mixed precision (halves activation memory; not used for the published runs).
<code>--cfg-options train_data_loader.num_workers=16</code>	Override any config key.
<code>--cfg-options randomness.seed=0</code> <code>randomness.deterministic=True</code>	Fixed seed and deterministic cuDNN.

Outputs in the work directory: <timestamp>/<timestamp>.log, vis_data/scalars.json (loss and metric curves), iter_*.pth, best_mIoU_iter_*.pth, and a copy of the resolved configuration.

6.3 Expected resources (paper, TITAN RTX 24 GB, batch 2)

Variant	Parameters	FLOPs (1024 ²)	Training time (100k it.)	Validation mIoU
U-MV-t	35.41 M	0.144 T	9.33 h	87.91 %
U-MV-s	54.24 M	0.215 T	11.8 h	88.02 %
U-MV-b	not reported	0.396 T	18.12 h	88.11 %

6.4 Practical notes

- **First run downloads the backbone.** The MambaVision code and weights are fetched from the Hugging Face Hub (nvidia/MambaVision-*-1K) when the model is built. Pre-download with tools/download_backbones.py for offline hosts.
- **DataLoader workers.** The base config uses 8 workers. The original runs used 42 on a 64 GB workstation; increase train_data_loader.num_workers if the GPU is starved (check the data_time field in the log).
- **WSL2 memory.** Large Mamba backbones can trigger CUDA system-memory fallback and freeze WSL2 when host RAM is exhausted. Mitigations: disable the system fallback in the NVIDIA Control Panel (*CUDA – System Fallback Policy* → *Prefer No System Fallback*), keep .wslconfig memory below physical RAM, and drop the page cache between runs (sync; echo 3 > /proc/sys/vm/drop_caches from the WSL2 shell as root).
- **Monitoring.** tail -f work_dirs/<name>/<timestamp>/<timestamp>.log, or add vis_backends=[dict(type='LocalVisBackend'), dict(type='TensorboardVisBackend')] to configs/_base_/default_runtime.py (requires pip install tensorboard).

7 Evaluation

7.1 Metrics

IoUMetric with `iou_metrics=['mIoU', 'mFscore']` reports, per class and averaged over background and acacia, the intersection-over-union, F-score, precision and recall (Equations 2–7 of the paper). Averages are over the two classes (mIoU, mFscore); the acacia-class values (`IoU.acacia`, `Fscore.acacia`) are the quantities of ecological interest.

7.2 Commands

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88" # folder -> its best_mIoU_iter_*.pth
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2 # test
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability # 00D
python tools/test.py configs/mambavision/U-MV-small.py work_dirs/U-MV-small --test-split test2 # own run
# released checkpoints (identical to the best_mIoU files; after git lfs pull)
for v in tiny small base; do
  python tools/test.py configs/mambavision/U-MV-$v.py \
    Pretrained_Weights/U-MV-${v}_latest.pth --test-split test2
done
```

Without `--test-split` the config's default split `img_dir/test` is used (the archive names it `test2`).

Options:

Option	Effect
<code>--test-split NAME</code>	Evaluate on <code>img_dir/NAME</code> + <code>ann_dir/NAME</code> (added in this repository).
<code>--work-dir DIR</code>	Where the JSON metrics and log are written.
<code>--out DIR</code>	Save predicted index masks for offline analysis.
<code>--show-dir DIR</code>	Save colour overlays (input, ground truth, prediction).
<code>--tta</code>	Multi-scale (0.5–1.75) + horizontal-flip test-time augmentation (slower; not used in the paper).

Evaluation uses the config's sliding-window mode (1024×1024 window, stride 512).

7.3 Reference results (paper, Table 1)

Model	Validation mIoU	Validation mF-score	Test mIoU	Test mF-score
U-MV-t	87.91	93.20	85.44	91.61
U-MV-s	88.02	93.27	85.38	91.58
U-MV-b	88.11	93.32	85.30	91.52

Validation covers ~ 6.5 km² (2 407 archived tiles) and the independent test set ~ 6.7 km² (3 123 tiles, split `test2`). On the generalisability set (~ 11 km², 2 162 tiles) U-MV-s reached 89.48 % mIoU and 94.17 % mF-score (§4.4 of the paper). A freshly evaluated released checkpoint should reproduce the published values within about ± 0.1 percentage points; larger deviations indicate a data or preprocessing mismatch (band order, mask encoding, image suffix) rather than model drift.

To confirm that a checkpoint matches a config, `python tools/inspect_checkpoint.py <file or work dir>` prints the variant inferred from the decoder shapes together with the iteration and the MMEngine version stored in the checkpoint.

8 Regional inference on orthomosaics

`tools/geospatial_inference.py` (one image) and `tools/batch_geospatial_inference.py` (folder tree) share the implementation in `umv/inference/`. They turn a georeferenced orthomosaic into a GeoPackage (or Shapefile) of crown polygons with per-polygon attributes, without ever holding the full raster in memory.

8.1 Processing steps

- 1 **Staging (optional, `--scratch-dir`).** The input is copied to a fast local directory. This is recommended on WSL2/Docker when the data reside on a Windows drive (`/mnt/<drive>`), where random window reads are slow.
- 2 **Tiling.** An edge-aligned grid of `--tile-size` windows with `--overlap` pixels is generated; the last tile of each row/column ends exactly at the raster border, so every tile has full context.
- 3 **Prediction.** Tiles are normalised as in training (ImageNet mean/std, RGB) and passed through `EncoderDecoder.encode_decode`, yielding logits at tile resolution; the softmax probability of the acacia class is kept.
- 4 **Blending.** `--blend center` (default) writes only the region between the mid-points of neighbouring overlaps, so every pixel is predicted exactly once from the tile in which it lies farthest from the border (seam-free by construction). `--blend hann` averages all overlapping predictions with a 2-D Hann weight.
- 5 **Probability raster.** Accumulators are finalised into a tiled, compressed GeoTIFF in the CRS of the input (`--save-prob` keeps it; `--prob-dtype uint8` stores 0–255).
- 6 **Vectorisation.** The raster is thresholded at `--thresh`, polygonised with GDAL `Polygonize` (streaming, 4- or 8-connectivity), filtered by `--min-area` (CRS units, m² for projected CRS) and enriched with `area` and `mean_prob` (mean probability inside each polygon, for threshold tuning in GIS).

8.2 Commands

```
python tools/geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py \
  --checkpoint Pretrained_Weights/U-MV-small_latest.pth \
  --input /data/orthos/Fujairah_block12_cog.tif \
  --output /data/predictions/Fujairah_block12_crowns.gpkg \
  --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob
```

```
python tools/batch_geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py \
  --checkpoint Pretrained_Weights/U-MV-small_latest.pth \
  --input-dir /data/orthos --output-dir /data/predictions \
  --scratch-dir /tmp/geospatial_work --skip-existing --continue-on-error
```

The batch script mirrors the input folder structure, skips finished outputs (`--skip-existing`, resumable) and writes `batch_summary.json`.

8.3 Parameters

Parameter	Default	Guidance
<code>--tile-size</code>	1024	Training tile size; keep.
<code>--overlap</code>	256	≥ 128 . Larger overlap = more context at the write boundary, more compute.
<code>--blend</code>	center	<code>hann</code> is marginally smoother on ambiguous crowns at a ~30 % higher cost.
<code>--batch-size</code>	8	16 fits on 24 GB GPUs in fp16 for U-MV-s; reduce on OOM.
<code>--precision</code>	fp16	CUDA autocast; fp32 for reference comparisons.

Parameter	Default	Guidance
--thresh	0.35	Probability threshold used for the regional maps. Tune per region with <code>mean_prob</code> ; 0.5 is the argmax-equivalent.
--min-area	0	e.g. <code>1 0</code> m ² removes spurious specks; the mean crown is ~17.5 m ² .
--connectivity	4	8 merges diagonally touching pixels.
--bands	1 2 3	Band indices fed as R, G, B (for 4-band RGBA orthomosaics <code>1 2 3</code> is still correct).
--band-order	rgb	Do not set <code>bgr</code> for standard orthomosaics; see below.
--prob-dtype	uint8	<code>float32</code> for full-precision probabilities (4× larger).
--num-workers	4	Raster reading processes.
--mp-context	(torch default)	<code>spawn</code> if forked workers misbehave with GDAL.

8.4 Channel order

During training, `LoadImageFromFile` reads TIFFs with OpenCV (BGR order) and `SegDataPreProcessor(bgr_to_rgb=True)` converts them to RGB before normalisation; the network therefore expects **RGB**. Rasterio returns bands in file order, which is RGB for standard orthomosaics, so the inference pipeline applies no channel swap. The previous versions of the scripts swapped the channels whenever `bgr_to_rgb=True` was set, which fed BGR tiles to the network; `--band-order bgr` reproduces that behaviour only for comparison.

8.5 Outputs

File	Content
<code><name>.gpkg</code> (layer <code><name></code>) or <code><name>.shp</code>	Crown polygons; attributes <code>area</code> (CRS units), <code>mean_prob</code> (0–1).
<code><name>_prob.tif</code> (with <code>--save-prob</code>)	Single-band probability raster; 0–255 for <code>uint8</code> .
<code>batch_summary.json</code> (batch)	Per-image tiles, polygons, timings, errors.

Post-processing in GIS: select polygons by `mean_prob`, dissolve touching crowns if instance separation is not required, and intersect with survey-zone boundaries.

8.6 Throughput

On a TITAN RTX with U-MV-s, fp16 and batch 8, throughput is roughly 4–6 tiles of 1024×1024 per second ($\approx 25\text{--}40 \text{ ha min}^{-1}$ at 2.5 cm GSD), dominated by raster I/O when reading from a Windows-mounted drive; staging to `/tmp` removes that bottleneck.

9 Reproducibility and paper-to-code mapping

9.1 Mapping of Section 3 of the paper to the configuration

Paper statement	Implementation
MambaVision T/S/B encoder, ImageNet-1K initialisation via timm/Hugging Face (§3.3, §3.5)	<code>MambaVisionBackbone(variant=..., pretrained=True) → AutoModel.from_pretrained('nvidia/MambaVision-{T,S,B}-1K', trust_remote_code=True)</code>
Encoder widths (80,160,320,640) / (96,192,384,768) / (128,256,512,1024) (§3.3)	<code>decode_head.encoder_channels</code> in <code>configs/mambavision/U-MV-*.py</code>
Lightweight CNN decoder with skip connections (§3.3)	<code>GenericUNetHead</code> , widths (256,128,64,32)
$L = L_{CE} + 3 L_{Dice}$ (Eq. 1)	<code>loss_decode=[CrossEntropyLoss(1.0), DiceLoss(3.0)]</code>
AdamW, $\beta=(0.9,0.999)$, weight decay 0.05 (§3.5)	<code>optimizer</code> in <code>_base_/schedules/schedule_100k_adamw.py</code>
Polynomial decay, power 0.9 (§3.5)	<code>PolyLR(power=0.9, eta_min=0)</code>
100 000 iterations, validation every 5 000 (§3.5, §4.2)	<code>train_cfg.val_interval=5000</code>
Batch size 2, tiles 1024×1024 (§3.2, §3.5)	<code>train_data_loader.batch_size=2, crop_size=(1024,1024)</code>
Gradient clipping (§3.5)	<code>clip_grad=dict(max_norm=0.01, norm_type=2)</code>
Random crop, horizontal flip, photometric perturbation (§3.5)	<code>train_pipeline</code>
Best validation checkpoint retained (§3.5)	<code>CheckpointHook(save_best='mIoU')</code>
Metrics: precision, recall, mIoU, mF-score (§3.6)	<code>IoUMetric(iou_metrics=['mIoU', 'mFscore'])</code>
Docker on a TITAN RTX workstation, 64 GB RAM (§3.5)	<code>docker/Dockerfile</code> (PyTorch 2.6.0 + CUDA 11.8 base); the original container definition is archived as <code>docs/reference/Dockerfile.original</code>

9.2 Reconciliation with the original training logs

The MMSegmentation work directories of the three published runs were recovered in September 2026 and their logs compared with the paper and the released configurations.

- Learning rate.** All three runs used a base learning rate of 1×10^{-4} with a backbone multiplier of 0.1; the logs list every encoder parameter at 1×10^{-5} . The paper's "initial learning rate of 1×10^{-5} " therefore refers to the encoder rate. The configs in this repository reproduce the logged setting.
- Schedule of the tiny variant.** The tiny log shows 100 000 iterations with validation every 5 000, identical to small and base, and reports the best validation mIoU (87.91 %, as in Table 1 of the paper) at iteration 100 000. The tiny config released earlier (lr 1×10^{-5} , 150 000 iterations) did not correspond to the run and has been aligned with the log.
- Checkpoints.** The released files are byte-identical (SHA-256) to `best_mIoU_iter_100000.pth` (tiny), `best_mIoU_iter_95000.pth` (small) and `best_mIoU_iter_60000.pth` (base) of the work directories, i.e. the best-validation checkpoints used for the paper's test results.
- Seeds.** No seed was fixed; runs are not bit-reproducible. For controlled comparisons add `--cfg-options randomness.seed=0 randomness.deterministic=True`.
- MMSegmentation revision.** The original container installed the main branch (`docs/reference/Dockerfile.original`); v1.2.2 is the last tagged release and is API-identical for the components used here.
- Channel order at inference.** The original inference scripts swapped channels to BGR before the network; the revised pipeline feeds RGB, which is what the training preprocessor produced

(docs/05_inference.md, §5.4). Regional maps produced with the old scripts may therefore differ slightly; tools/test.py is unaffected and provides the reference.

- 7 **Training split of the archived dataset.** The archived train folder holds 4 893 tiles (1024×1024 , identical in every copy found), whereas the paper reports 26 615. The tile indices run from 0 to 19 765 with gaps, and some files carry modification dates after the training runs (August 2025), which indicates that the training folder was pruned after the models were trained. Validation (2 407; the logs evaluate 1 204 batches of 2), test (3 123) and generalisability (2 162) tiles are complete. Consequently, evaluation and inference with the released checkpoints are fully reproducible from the archive, whereas retraining from the archive reproduces the recipe but not the published model.

9.3 What is and is not fixed by the pinned environment

Fixed: library versions, CUDA kernels, preprocessing, architecture, optimiser and schedule. Not fixed: cuDNN algorithm selection (cudnn_benchmark=True), data-loading order without a seed, and the exact Hugging Face revision of the MambaVision weights (pin a revision= in MambaVisionBackbone via model_name if the Hub repository is updated upstream).

9.4 Archival

The code is archived on Zenodo (DOI 10.5281/zenodo.18068379). When re-archiving this revision, include docs/reference/environment.frozen.yml, the Docker image digest (docker images --digests umv), and the SHA-256 of the checkpoints listed in Pretrained_Weights/README.md.

10 Troubleshooting

Symptom	Cause	Remedy
ModuleNotFoundError: No module named 'mmdcv._ext'	mmdcv-lite or a wheel built for another PyTorch/CUDA is installed.	Reinstall MMCV from source against the active PyTorch (docs/01_installation.md, step 2) or use the Docker image.
No module named 'mamba_ssm' / selective_scan_cuda import error	Kernels not built or built for a different PyTorch.	<code>pip install --no-build-isolation mamba-ssm==2.2.4</code> with nvcc available; confirm with <code>tools/verify_install.py</code> .
KeyError: 'MambaVisionBackbone is not in the mmseg::model registry'	umv not imported.	Configs must contain <code>custom_imports = dict(imports=['umv'])</code> (all shipped configs do); run scripts from the repository root or <code>pip install -e ..</code>
OSError: We couldn't connect to 'https://huggingface.co'	Backbone download blocked (proxy, offline).	Pre-download on a connected machine (<code>tools/download_backbones.py</code>), copy <code>hf_cache/</code> and set <code>HF_HUB_OFFLINE=1</code> .
UnpicklingError: invalid load key, 'v' when loading a .pth	The file is a Git LFS pointer.	<code>git lfs install && git lfs pull</code> (see Pretrained_Weights/README.md).
_pickle.UnpicklingError: Weights only load failed	PyTorch ≥ 2.6 with an old MMEEngine.	Use MMEEngine 0.10.7 (pinned).
size mismatch for decode_head.decoder_stages.0.0.weight	Checkpoint and config variant differ.	<code>python tools/inspect_checkpoint.py <ckpt></code> prints the matching config.
Validation mIoU ≈ 50 % or all-background predictions	Masks are 0/255 or RGB, or wrong suffix.	Re-encode masks as 0/1 uint8 single band (docs/02_data_preparation.md, §2.3).
Test mIoU several points below the paper	Band order or resolution mismatch.	Ensure RGB orthomosaics, 2.5–3 cm GSD, no <code>--band-order bgr</code> .
CUDA out of memory during training	1024 ² tiles with batch 2 need ~20 GB on U-MV-b.	Use <code>--amp</code> , or <code>train_dataloader.batch_size=1</code> with doubled iterations.
WSL2 freezes; GPU memory shows 24 GB + system RAM use	CUDA system fallback exhausting host RAM.	Set <i>Prefer No System Fallback</i> in the NVIDIA Control Panel; reduce batch size; drop page cache between runs.
Docker build fails at the MMCV step with cannot allocate memory / Failed to build mmdcv	too many parallel nvcc jobs for the RAM given to Docker/WSL2	rebuild with <code>--build-arg MAX_JOBS=2 --build-arg CUDA_ARCH="7.5"</code> (8.6 for RTX A5000); raise <code>memory=</code> in <code>.wslconfig</code> ; cached layers are reused
RuntimeError: DataLoader worker ... Bus error / shared-memory errors in Docker	/dev/shm too small.	Run with <code>--ipc=host</code> (compose file does) or <code>--shm-size=16g</code> .
osgeo missing in inference	GDAL Python bindings absent.	Install via conda-forge; the rasterio fallback only handles rasters ≤ 1.5 gigapixels.
Straight cut lines in the crown map	<code>--overlap</code> below ~128 or <code>--blend</code> misconfigured.	Keep the defaults (overlap 256, centre-crop).
Very slow inference from /mnt/<drive>	Windows filesystem access from WSL2.	Use <code>--scratch-dir /tmp/geospatial_work</code> and COG inputs.
img_ratios NameError when loading a config	Legacy dataset config.	Fixed in this revision; <code>img_ratios</code> is defined in <code>configs/_base_/datasets/uav_acacia_dataset.py</code> .

A Revision notes (September 2026)

This revision prepares the repository for hand-over and independent reproduction. The network architecture, released configurations and checkpoints are unchanged; the changes concern packaging, correctness of the auxiliary code, portability and documentation.

A.1 Defects found in the previous revision

#	Defect	Effect	Resolution
1	<code>mmseg/dataset/ade.py</code> (singular <code>dataset</code>) was meant to overwrite <code>MMSegmentation</code> 's <code>mmseg/datasets/ade.py</code> ; the copy-in instructions never placed it there.	Configs referenced <code>ADE20KDataset</code> , which expects <code>.jpg</code> images and 150 classes; a fresh installation could not load <code>.tif</code> tiles or report two-class metrics.	New registered dataset <code>UAVAcaciaDataset</code> in <code>umv/datasets/</code> ; no upstream file is modified.
2	<code>img_ratios</code> undefined in the dataset config.	<code>Config.fromfile</code> raised <code>NameError</code> on every config.	Defined.
3	<code>configs/_base_/default_runtime.py</code> absent from the repository.	Configs unloadable outside a full <code>MMSegmentation</code> checkout.	Added (verbatim from v1.2.2).
4	Inference scripts swapped <code>RGB</code> → <code>BGR</code> whenever <code>bgr_to_rgb=True</code> .	Network received <code>BGR</code> tiles although it was trained on <code>RGB</code> .	Removed; <code>--band-order bgr</code> retained for comparison.
5	Batch script used <code>mode='whole'</code> with <code>ori_shape=(h, w)</code> for border tiles.	Padded border tiles were resized to the valid size, misaligning predictions along raster edges.	Both scripts now use <code>encode_decode</code> at tile resolution and crop.
6	Centre-crop writer double-wrote border regions when a raster edge fell inside the first tile of a row.	Silent averaging at borders.	Edge-aligned grid with mid-point write boundaries; property-tested (<code>tests/test_tiling.py</code>).
7	Minimum-area filtering documented in the README but not implemented.	—	<code>--min-area</code> implemented; <code>area</code> attribute added.
8	Hard-coded paths, thresholds and device settings inside the scripts.	Edits required for every run.	<code>argparse</code> CLI with documented defaults.
9	<code>num_workers=42</code> in the dataset config.	Failure or thrashing on smaller hosts.	8 (documented).
10	Backbone always downloaded ImageNet weights, even when loading a fine-tuned checkpoint; no offline path.	Network access mandatory at inference time.	<code>pretrained=False</code> when a checkpoint is given; <code>HF_HUB_OFFLINE</code> / <code>local_files_only</code> supported; <code>tools/download_backbones.py</code> .
11	<code>environment.yml</code> was a full conda dump of the container's base environment (prefix: <code>/opt/conda</code> , 300 packages).	Not recreatable; misleading as a specification.	Moved to <code>docs/reference/environment.frozen.yml</code> ; replaced by <code>requirements/*.txt</code> , <code>pyproject.toml</code> and <code>docker/Dockerfile</code> .
12	<code>Assets/mamba_vision_architecture.png</code> was 92 MB ($14\,639 \times 14\,198$ px).	Slow clones, README rendering issues.	Downscaled to 2 400 px (2.5 MB); the original remains in Git history.
13	<code>.idea/workspace.xml</code> committed.	IDE noise.	Removed and ignored.
14	<code>torch==4.65.2</code> , <code>torch==2.6.0+cu118</code> pins in <code>requirements.txt</code> not installable from PyPI without extra index.	<code>pip install -r requirements.txt</code> failed.	Split requirements; <code>PyTorch/MMCV/mamba-ssm</code> installed explicitly.

A.2 Structural changes

- `umv/` installable package (`pip install -e .`) with `models/`, `datasets/`, `inference/`, `checkpoint.py`. Registered names `GenericUNetHead`, `mamba_{tiny,small,base}_vision_timm` are kept, and `MambaVisionBackbone` is added, so configs embedded in old checkpoints still resolve.
- Module attribute names (`backbone.backbone.*`, `decode_head.decoder_stages.*`, `decode_head.segmentation_head.*`, `decode_head.conv_seg.*`) are preserved; released checkpoints load without key remapping.
- Configs split into `_base_/models`, `_base_/datasets`, `_base_/schedules`, `_base_/default_runtime.py`; variant files contain only variant-specific fields. Numerical hyper-parameters are unchanged.
- `tools/train.py` and `tools/test.py` vendored from `MMSegmentation v1.2.2` (plus `--test-split`), so the repository is self-contained.
- New tools: `verify_install.py`, `inspect_checkpoint.py`, `download_backbones.py`, `batch_geospatial_inference.py` (renamed from `Batch_processing_geospatial_inference.py`).
- `tests/` (14 tests) covering tiling, blending, vectorisation, checkpoint variant detection and the end-to-end inference pipeline on synthetic GeoTIFFs (CPU).

A.3 Compatibility with the archived work directories

`umv/compat.py::load_config` detects configs that import `mmseg.custom_models.*` (the archived training configs) and maps them to the new package, including `ADE20KDataset` → `UAVAcaciaDataset`. All tools use it, so `tools/test.py` `<archived config>` `<archived checkpoint>` works without edits. `docker/.env` (`DATA_DIR`, `WEIGHTS_DIR`) decouples the container from the location of the hand-over folder.

A.4 Validation performed in this revision

- All three configs load; models build with a stub encoder and run a training step (CE + Dice losses), `encode_decode`, and sliding-window prediction on CPU (MMEEngine 0.10.7, MMSegmentation 1.2.2).
- The inference pipeline recovers synthetic crowns with correct CRS, areas ($\pm 5\%$), attributes and seam-free tiling for both blending modes.
- The Git LFS objects of the three released checkpoints are present on GitHub and their SHA-256 values equal those of `best_mIoU_iter_{100000,95000,60000}.pth` in the `tiny/small/base` work directories.
- The original container definition was recovered and archived as `docs/reference/Dockerfile.original`; it uses the same base image and MMCV build route as `docker/Dockerfile`.

A.5 Not validated here (requires a GPU host)

- The Docker image build (MMCV and mamba-ssm compilation) and the Hugging Face backbone instantiation; the Hub was unreachable from the sandbox in which this revision was prepared. Run `docs/08_handover_checklist.md` steps 7–12 on the workstation and report any deviation.
- Numerical equivalence of the released checkpoints with the paper's metrics.

A.6 Resolved from the recovered work directories (September 2026)

- The tiny configuration has been aligned with its training log (lr 1e-4, 100 000 iterations); the previously released tiny config did not match the run.
- The released checkpoints are the best-validation checkpoints (SHA-256 verified against `best_mIoU_iter_{100000,95000,60000}.pth`).

- The archived training split (4 893 tiles) is a pruned subset of the 26 615 tiles used for the published models; no fuller copy exists on the project disks. Evaluation and inference are exact; retraining reproduces the recipe.

A.7 Decisions left to the authors

- 1 Whether to publish the checkpoints on Zenodo/Hugging Face in addition to Git LFS (bandwidth quota of GitHub LFS is 1 GB month⁻¹ on free plans; three full downloads exhaust it).
- 2 Whether regional maps produced with the earlier BGR-swapped scripts should be regenerated.

B Pretrained weights

The three released checkpoints are stored with **Git LFS**. A plain `git clone` without LFS yields 134-byte pointer files, not weights. Fetch the binaries with:

```
git lfs install
git lfs pull # all three (~815 MB)
git lfs pull --include="Pretrained_Weights/U-MV-small_latest.pth" # one variant
```

Variant	File	Size	SHA-256 (LFS oid)
U-MV-tiny	U-MV-tiny_latest.pth	159 MB	85796b37582647a6d83973e240ebf30e0a562676bd44ae84eb81c953dc8900ac
U-MV-small	U-MV-small_latest.pth	234 MB	0c8d2b3dcd6f6272ea12d9fcdac2cba79ca185277cb2f5d1eff55e5d065ed9d8
U-MV-base	U-MV-base_latest.pth	422 MB	fcd86b17fb1c49e4f9555538526cc0edff805d0303463bbcc9336b7e537b66d3

Verify a download with `sha256sum <file>` and identify the variant of any local checkpoint with:

```
python tools/inspect_checkpoint.py /path/to/checkpoint.pth
```

A checkpoint obtained elsewhere (e.g. a `best_mIoU_iter_*.pth` from a `work_dirs/` folder) can be used directly by passing its path to `tools/test.py` or the inference scripts; no renaming is required.

B.1 Provenance (verified September 2026)

The released files are byte-identical to the best-validation checkpoints of the original MMSegmentation work directories (SHA-256 verified):

Released file	Work directory	Checkpoint	Iteration
U-MV-tiny_latest.pth	mambavision-t_generic-unet_acacia	best_mIoU_iter_100000.pth	100 000
U-MV-small_latest.pth	mambavision-s_generic-unet_acacia-88	best_mIoU_iter_95000.pth	95 000
U-MV-base_latest.pth	mambavision-b_generic-unet_acacia	best_mIoU_iter_60000.pth	60 000

The `_latest` suffix therefore denotes the latest release, not the last training iteration. `iter_100000.pth` files in the work directories are the final-iteration checkpoints and were not released.

B.2 Original work directories

Passing a work-directory folder to any tool selects its `best_mIoU_iter_*.pth`:

```
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
python tools/test.py configs/mambavision/U-MV-small.py \
    "/weights/mambavision-s_generic-unet_acacia-88" --test-split test2
```

C Command reference

C.1 Windows (PowerShell, in D:\U-MV)

```
tools\windows\copy_archive.cmd D:\A.tortilis_Data_Model # copy dataset + weights from Z:, check counts
copy docker\.env.windows.example docker\.env # then edit the two paths if needed
docker\umv.cmd gpu # GPU visible to Docker?
docker\umv.cmd build 7.5 # image build; 7.5 = TITAN RTX, 8.6 = RTX A5000
docker\umv.cmd shell # interactive container; exit to leave
```

C.2 Environment (Linux / WSL2)

```
cp docker/.env.example docker/.env # set DATA_DIR and WEIGHTS_DIR
docker compose --env-file docker/.env -f docker/docker-compose.yml build
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
python tools/verify_install.py --variant small # stack check + forward pass
python tools/download_backbones.py # cache backbones; then export HF_HUB_OFFLINE=1
python -m pytest tests -q # unit tests (CPU)
```

C.3 Data and checkpoints

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
```

C.4 Training

```
python tools/train.py configs/mambavision/U-MV-small.py
python tools/train.py configs/mambavision/U-MV-small.py --resume --work-dir work_dirs/U-MV-small
python tools/train.py configs/mambavision/U-MV-small.py \
  --cfg-options randomness.seed=0 randomness.deterministic=True
```

C.5 Evaluation

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88" # folder resolves to best_mIoU_iter*.pth
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2 \
  --out preds/ --show-dir vis/
```

C.6 Regional inference

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
  --input /data/orthos/block12_cog.tif \
  --output /data/predictions/block12_crowns.gpkg \
  --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob

python tools/batch_geospatial_inference.py \
  --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" \
  --input-dir /data/orthos --output-dir /data/predictions \
  --scratch-dir /tmp/geospatial_work --skip-existing --continue-on-error
```

C.7 Orthomosaic preparation

```
gdal_translate -of COG -co COMPRESS=DEFLATE -co BLOCKSIZE=512 \
  -co NUM_THREADS=ALL_CPUS in.tif out_cog.tif
```